Course Code: CSE 499B
Section: 13
Project Group No: 06
Group Members:
Md.Murad Ul Momin (ID: 2232360642)
Rima Saha (ID: 2132517042)
Sakia Tabassum Ananna (ID: 2221433042)
Ashraful Hossain Riyan (ID: 2232845642)

PRIVATE SHIELD: Weekly Update 3

I. INTRODUCTION

In this week connected already-prepared client datasets to the existing federated-learning framework. The software now checks client inputs, runs configurable FedAvg, evaluates validation data each round, and exports the final global model. It reuses the configurable PyTorch MLP, clients, server, and sample-weighted aggregation from Milestone A. Verification used synthetic fixtures and artificial updates without fitting a model; no research results were produced.

II. IMPLEMENTATION WORK

A. Model Export and Configuration

artifact.py now provides save_model_artifact(), saving compatible weights with the original model/data contract and purpose flags. It rejects invalid inputs and preserves existing files. training_config.py validates the prepared-FedAvg YAML, resolves paths, and supports one shared random initialization or a compatible saved-model warm start. The approved export policy is the final round, independent of validation scores. A PyTorch stacking-model checkpoint (pytorch_stacking_model.pth) was also provided for future integration and evaluation.

B. Federated Inputs and Split Evidence

federated_bundle.py collects named client training partitions and separate server validation. It checks roles, client IDs, compatible contracts, and stable file hashes. split_checks.py reads separate JSON record/group ID files, keeping existing NPZ files unchanged, and rejects declared overlap. These checks depend on truthful original IDs; they cannot prove data provenance, correct preprocessing, or final-test isolation. An independent real-data audit is still required.

C. Execution and Validation Logging

federated_runner.py connects checked inputs, seeds, shared initialization, and the existing FL components. main.py adds --federated --config. experiment_logger.py labels validation scores and records settings, source hashes, client contributions, and metrics. Model export precedes the completion summary. Final-test inputs are excluded from training and evaluated separately through the prepared workflow. Normal federated execution performs local training.

III. VERIFICATION AND CURRENT STATUS

The recorded selected regression passed 686 cases; six existing optimization/backpropagation cases were excluded. An additional exact-template check passed. Runner tests replaced local training with known tensor changes and forbade optimizer/backward execution. Checks covered weighted aggregation, shared broadcasts, compatibility, validation labels, repeatability, final-round export, and reuse through prediction/evaluation. Week 3 infrastructure is complete; real-data readiness, numerical learning behavior, and CUDA execution remain unverified.

FUTURE OBJECTIVES

- Inspect real data and agree feature/class mappings, client partitions, preprocessing, and independent split auditing.
- After data-readiness checks, run a small FedAvg training pilot, then establish a clean real-data baseline.
- Implement FedMADE, SecureDyn-inspired trust, poisoning, and the proposed method in later agreed stages.